COMSATS University Islamabad

Attock Campus

Department Of Computer Science

Course	***IS(LAB)***
Instructor	***Ms. Ambreen Gul***
Assignment No.	***03***

Student Details

Registration No.	***Name***
FA24-BSE-013	***Ahmad Raza***

Task 1:

Code Screenshots

```python
import hashlib
import datetime
# Create a Block class
class Block:
    def __init__(self, index, timestamp, data, previous_hash):
        self.index = index                  # Block number
        self.timestamp = timestamp          # Time of creation
        self.data = data                    # Data stored in block
        self.previous_hash = previous_hash  # Hash of previous block
        self.hash = self.calculate_hash()   # Current block hash
    # Function to calculate hash using SHA256
    def calculate_hash(self):
        block_string = str(self.index) + str(self.timestamp) + str(self.data) + str(self.previous_hash)
        return hashlib.sha256(block_string.encode()).hexdigest()
# Create Blockchain class
class Blockchain:
    def __init__(self):
        self.chain = [self.create_genesis_block()]  # Initialize with first block
    # Create the first block (Genesis Block)
    def create_genesis_block(self):
        return Block(0, datetime.datetime.now(), "Genesis Block", "0")
    # Get last block in chain
    def get_last_block(self):
        return self.chain[-1]
    # Add new block
    def add_block(self, data):
        last_block = self.get_last_block()
        new_block = Block(
            index=last_block.index + 1,
            timestamp=datetime.datetime.now(),
            data=data,
            previous_hash=last_block.hash
        )
        self.chain.append(new_block)
# Function to display blockchain
def print_chain(blockchain):
    for block in blockchain.chain:
```

```python
# Function to display blockchain
def print_chain(blockchain):
    for block in blockchain.chain:
        print("Index:", block.index)
        print("Timestamp:", block.timestamp)
        print("Data:", block.data)
        print("Previous Hash:", block.previous_hash)
        print("Hash:", block.hash)
        print("-" * 50)
# --------------- MAIN PROGRAM ---------------
# Create blockchain
my_blockchain = Blockchain()
# Add 4–5 blocks
my_blockchain.add_block("Block 1 Data")
my_blockchain.add_block("Block 2 Data")
my_blockchain.add_block("Block 3 Data")
my_blockchain.add_block("Block 4 Data")
print("🔗 Original Blockchain:\n")
print_chain(my_blockchain)
# --------------- MODIFY A BLOCK ---------------
print("\n Modifying Block 2 Data...\n")
# Change data of block 2
my_blockchain.chain[2].data = "Hacked Data"
my_blockchain.chain[2].hash = my_blockchain.chain[2].calculate_hash()
print(" Blockchain After Modification:\n")
print_chain(my_blockchain)
```

Explanation of Code

Block Class

This program builds a very simple version of a blockchain to help you understand how real blockchains work. It starts by defining a **Block class**, which represents a single unit in the chain. When a block is created, it stores an index (its position), a timestamp (when it was created), some data (any information you want to store), the hash of the previous block, and its own hash. The hash is generated using the calculate_hash() function, which combines all the block's information into a single string and passes it through the SHA256 algorithm. This ensures that every block has a unique digital fingerprint. The reason we create this function separately is to keep the logic reusable and clean—any time the block's data changes, we can simply call this function again to recalculate the hash instead of rewriting the hashing logic multiple times.

Blockchain Class

The Blockchain class is created to manage all the blocks. When a blockchain object is initialized, it automatically creates the first block called the "genesis block" using the create_genesisblock() function. This function exists because every blockchain must start somewhere, and the first block doesn't have a previous block to refer to, so we manually assign it a default previous hash (usually "0"). The get_last_block() function is used to easily access the most recent block in the chain, which is important because every new block must link to the previous one. Instead of repeatedly writing code to find the last block, this function simplifies the process and improves readability.

Add Block Function

The add_block() function is responsible for adding new blocks to the blockchain. It first retrieves the last block, then creates a new block with an incremented index, the current timestamp, the provided data, and the previous block's hash. This function ensures that all blocks are properly linked together, maintaining the structure of the blockchain. We create this function to automate block creation and enforce consistency—without it, we would have to manually set indexes and hashes every time, which could lead to errors.

Print Chain Function

The print_chain() function is used to display all blocks in a readable format. It loops through each block in the chain and prints its details. This function is created to separate display logic from core blockchain logic, making the code cleaner and easier to understand or modify later.

In the main part of the program, we create a blockchain and add several blocks to it. Then, to demonstrate how blockchain security works, we manually modify the data of one block and recalculate its hash. This change breaks the chain because the next block still holds the old hash value, showing how even a small modification affects the entire system. This illustrates the key idea behind blockchain: data integrity is protected because every block depends on the one before it.

Ouput Screenshots

```
PS D:\BS-SE 04\SE-4-VScode> & C:\Users\DELL\AppData\Local\Python\bin\python.exe "d:/BS-SE 04/SE-4-VScode/IS Lab/Assignment_3.py"
🔗 Original Blockchain:

Index: 0
Timestamp: 2026-05-02 23:46:56.772562
Data: Genesis Block
Previous Hash: 0
Hash: a45916c81f51163a0d62ad9fb44ed503abcba75cca4b2a5b4765a226d4672e26
--------------------------------------------------
Index: 1
Timestamp: 2026-05-02 23:46:56.773298
Data: Block 1 Data
Previous Hash: a45916c81f51163a0d62ad9fb44ed503abcba75cca4b2a5b4765a226d4672e26
Hash: 27ffb1133df498d6dd71504c256516317a927627db928dae3f56e9be95aad8f7
--------------------------------------------------
Index: 2
Timestamp: 2026-05-02 23:46:56.773363
Data: Block 2 Data
Previous Hash: 27ffb1133df498d6dd71504c256516317a927627db928dae3f56e9be95aad8f7
Hash: 35eded34b2af63d08b98e55a37261dd9a4bb2c60c0092d5963fc0e6382bc5356
--------------------------------------------------
```

```
Index: 3
Timestamp: 2026-05-02 23:46:56.773382
Data: Block 3 Data
Previous Hash: 35eded34b2af63d08b98e55a37261dd9a4bb2c60c0092d5963fc0e6382bc5356
Hash: 787947a8a9f8e7c44f719c63cc301ac1855b9b533bb60cd36c1a0d72236342a2
--------------------------------------------------
Index: 4
Timestamp: 2026-05-02 23:46:56.773395
Data: Block 4 Data
Previous Hash: 787947a8a9f8e7c44f719c63cc301ac1855b9b533bb60cd36c1a0d72236342a2
Hash: d3288b1ddb2eccc2338ae02766dfba1dd6bcab91c961888ee670a28f8373ea4f
--------------------------------------------------
```

```
Modifying Block 2 Data...

Blockchain After Modification:

Index: 0
Timestamp: 2026-05-02 23:46:56.772562
Data: Genesis Block
Previous Hash: 0
Hash: a45916c81f51163a0d62ad9fb44ed503abcba75cca4b2a5b4765a226d4672e26
--------------------------------------------------
Index: 1
Timestamp: 2026-05-02 23:46:56.773298
Data: Block 1 Data
Previous Hash: a45916c81f51163a0d62ad9fb44ed503abcba75cca4b2a5b4765a226d4672e26
Hash: 27ffb1133df498d6dd71504c256516317a927627db928dae3f56e9be95aad8f7
--------------------------------------------------
Index: 2
Timestamp: 2026-05-02 23:46:56.773363
Data: Hacked Data
Previous Hash: 27ffb1133df498d6dd71504c256516317a927627db928dae3f56e9be95aad8f7
Hash: 00852f33770fe438f88b0f3a68076aebb3c71fced3faeb5c0013293c941d0e63
```

```
Data: Hacked Data
Previous Hash: 27ffb1133df498d6dd71504c256516317a927627db928dae3f56e9be95aad8f7
Hash: 00852f33770fe438f88b0f3a68076aebb3c71fced3faeb5c0013293c941d0e63
--------------------------------------------------
Index: 3
Timestamp: 2026-05-02 23:46:56.773382
Data: Block 3 Data
Previous Hash: 35eded34b2af63d08b98e55a37261dd9a4bb2c60c0092d5963fc0e6382bc5356
Hash: 787947a8a9f8e7c44f719c63cc301ac1855b9b533bb60cd36c1a0d72236342a2
--------------------------------------------------
Index: 4
Timestamp: 2026-05-02 23:46:56.773395
Data: Block 4 Data
Previous Hash: 787947a8a9f8e7c44f719c63cc301ac1855b9b533bb60cd36c1a0d72236342a2
Hash: d3288b1ddb2eccc2338ae02766dfba1dd6bcab91c961888ee670a28f8373ea4f
--------------------------------------------------
PS D:\BS-SE 04\SE-4-VScode>
```

Task 2:

Code Screenshots

U | Task2.py U | task_3.py U | Activity_1.java U | CSP_colors U | Assignment_3.py U | Assignment_4.py U × | unitTestingQ1.py U | Q1.py

E-4-VScode > IS Lab > Assignment_4.py > ...

```python
def scalar_multiply(k, P):
    temp = P
    while k > 0:
        if k % 2 == 1:
            result = point_add(result, temp)
        temp = point_double(temp)
        k = k // 2
    return result
# -------- KEY GENERATION --------
private_key = 5   # small integer
public_key = scalar_multiply(private_key, G)
print("Private Key:", private_key)
print("Public Key:", public_key)
# -------- ENCRYPTION (EC-ElGamal simplified) --------
def encrypt(message_point, public_key):
    k = 3  # random small number
    C1 = scalar_multiply(k, G)
    C2 = point_add(message_point, scalar_multiply(k, public_key))
    return (C1, C2)
# -------- DECRYPTION --------
def decrypt(cipher, private_key):
    C1, C2 = cipher
    shared = scalar_multiply(private_key, C1)
    # Inverse of shared point
    x, y = shared
    inverse_shared = (x, (-y) % p)
    message = point_add(C2, inverse_shared)
    return message
# -------- TEST MESSAGE --------
# Represent message as a point (example)
message = (10, 7)
print("\nOriginal Message:", message)
cipher = encrypt(message, public_key)
print("Encrypted:", cipher)
decrypted = decrypt(cipher, private_key)
print("Decrypted:", decrypted)
```

xing completed. Ln 7, Col 11 Spaces: 4 UTF-8 CRLF { } Python

U | Task2.py U | task_3.py U | Activity_1.java U | CSP_colors U | Assignment_3.py U | Assignment_4.py U × | unitTestingQ1.py U | Q1.py

E-4-VScode > IS Lab > Assignment_4.py > ...

```python
# Simple ECC implementation (for learning only, not secure)
# Curve parameters: y^2 = x^3 + ax + b (mod p)
a = 2
b = 3
p = 97   # prime number (small for simplicity)
# Base point (generator)
G = (3, 6)
# Function to find modular inverse
def mod_inverse(k, p):
    # Fermat's Little Theorem
    return pow(k, p - 2, p)
# Function for point addition
def point_add(P, Q):
    if P == "O":  # Identity point
        return Q
    if Q == "O":
        return P
    x1, y1 = P
    x2, y2 = Q
    # If points are same → point doubling
    if P == Q:
        return point_double(P)
    # Calculate slope (lambda)
    m = ((y2 - y1) * mod_inverse(x2 - x1, p)) % p
    # Calculate new point
    x3 = (m*m - x1 - x2) % p
    y3 = (m*(x1 - x3) - y1) % p
    return (x3, y3)
# Function for point doubling
def point_double(P):
    if P == "O":
        return "O"
    x, y = P
    # slope formula
    m = ((3*x*x + a) * mod_inverse(2*y, p)) % p
    x3 = (m*m - 2*x) % p
    y3 = (m*(x - x3) - y) % p
```

xing completed. Ln 7, Col 11 Spaces: 4 UTF-8 CRLF { } Python

Explanation of Code

This program demonstrates the basic working of Elliptic Curve Cryptography (ECC) using a very small curve so that the logic is easy to follow. First, we define an elliptic curve equation of the form y2=x3+ax+bmod py^2 = x^3 + ax + b \mod py2=x3+ax+bmodp, where a, b, and p are constants. The value p is a small prime number used to keep all calculations within a finite field. A base point G is chosen on the curve, which acts as the starting point for generating keys. All operations in ECC revolve around manipulating points on this curve instead of working directly with numbers like in traditional cryptography.

Mod Inverse Function
The program includes a function mod_inverse() to compute the modular inverse, which is necessary for division in modular arithmetic. This function is created separately because division is not directly possible in modular math, so we must convert it into multiplication using inverses. Having this as a function makes the code reusable and avoids repeating complex logic multiple times.

Point Add Function
The point_add() function handles the addition of two different points on the curve. It calculates the slope between the two points and uses mathematical formulas to determine the resulting point. This function also checks if the two points are the same, in which case it calls the point_double() function instead. We create this function to clearly separate the logic of adding two distinct points, making the code easier to understand and maintain.

Point Double Function
The point_double() function is specifically used when a point is added to itself. Since the formula for doubling is different from normal addition, it is implemented separately. This separation improves clarity and ensures correctness, as combining both cases into one function would make the code more complex and harder to debug.

Scalar Multiply Function
The scalar_multiply() function is one of the most important parts of ECC. It performs repeated addition of a point (like multiplying a number by repeated addition). This is used to generate keys. Instead of adding a point again and again manually, this function uses an efficient method (double-and-add) to compute the result quickly. We create this function to automate and optimize repeated operations, which are essential in ECC.
For key generation, a private key is chosen as a small integer, and the public key is generated by multiplying the base point G with the private key using the scalar_multiply() function. This works because ECC is based on a one-way operation—it's easy to compute the public key from the private key, but very difficult to reverse the process.

Encrypt Function
The encrypt() function implements a simplified version of EC-ElGamal encryption. It takes a message (represented as a point on the curve) and the public key, then generates two ciphertext points using a random number k. This randomness ensures that even if the same message is encrypted multiple times, the output will be different. We create this function to encapsulate the

encryption process and keep the logic separate from the rest of the code.

Decrypt Function
The decrypt() function reverses the encryption process using the private key. It computes a shared secret from the ciphertext and removes it to recover the original message point. This function is created to clearly define the decryption process and demonstrate how the private key is used to retrieve the original data securely.

The program tests the implementation using a small message represented as a point. It encrypts the message and then decrypts it back to verify correctness. This shows the core idea of ECC: secure communication using mathematical operations on elliptic curves, where the private key remains secret while the public key is shared.

Output Screenshot

```
PS D:\BS-SE 04\SE-4-VScode> & C:\Users\DELL\AppData\Local\Python\bin\python.exe "d:/BS-SE 04/SE-4-VScode/IS Lab/Assignment_4.py"
Private Key: 5
Public Key: (91, 91)

Original Message: (10, 7)
Encrypted: ((80, 87), (64, 62))
Decrypted: (94, 60)
PS D:\BS-SE 04\SE-4-VScode>
```

The END